SKN35 4Team

팀 소개

최우석

Django 프로젝트 폴더 구조 설계
백엔드 구현 · API 엔드포인트 정의 · 발표

조현주

AWS Docker 배포 환경 구축
LLM 파이프라인 검증

손채영

LLM 파이프라인 검증
파이프라인 QA 수행

심성욱

화면 UI 구현 · 백엔드 API 연결 보조
발표자료 제작

SKN35 4Team · 4차 프로젝트

Langration

옛 LangChain 코드를 최신 버전으로 고치고, 근거와 검사 결과까지 보여 주는 웹 서비스

Django LangGraph TiDB Docker AWS EC2

2026-11-05 · 발표 최우석 · 배포 http://16.176.158.134/

AGENDA

발표 순서

01

3차 보완점

3차 기반으로 만든 4차

02

해결 방향

배포 설계 · 평가 범위

03

바뀐 기술

구성 · 분류 · 진행 표시

04

시연

배포 주소에서 바로

05

결과 정리

3차 대비 4차

01 · 3차 보완점

챗봇은 동작했지만, 서비스로는 부족했습니다

3차 Langration

옛 LangChain 코드를 근거로 1.x로 고치고, 11가지 검사 · 최대 3회 재시도로 확인하는 Streamlit 챗봇

배포

화면과 그래프가 한 앱에

Streamlit Cloud 하나에 화면 · 그래프 · 저장을 모두 실어 역할을 나누거나 옮기기 어려웠습니다

사용자

아이디만으로 로그인

비밀번호가 없어 아이디만 알면 남의 대화를 볼 수 있었고, 대화 삭제도 없었습니다

평가

코드 변환만 평가

문서 질문 답변은 재지 않았고, 최종 검증 통과는 53%에 그쳤습니다

02 · 해결 방향

배포를 새로 설계하고, 평가를 넓혔습니다

배포 설계

3차	Streamlit Cloud 앱 하나
4차	Django API 서버 + 화면 분리 Docker로 묶어 AWS EC2 배포 비밀번호 로그인 · 대화 삭제

평가 범위

3차	코드 변환 평가만
4차	코드 변환 평가 다시 측정 + 문서 질의응답 RAG 평가 (RAGAS) 분류 실패도 따로 기록

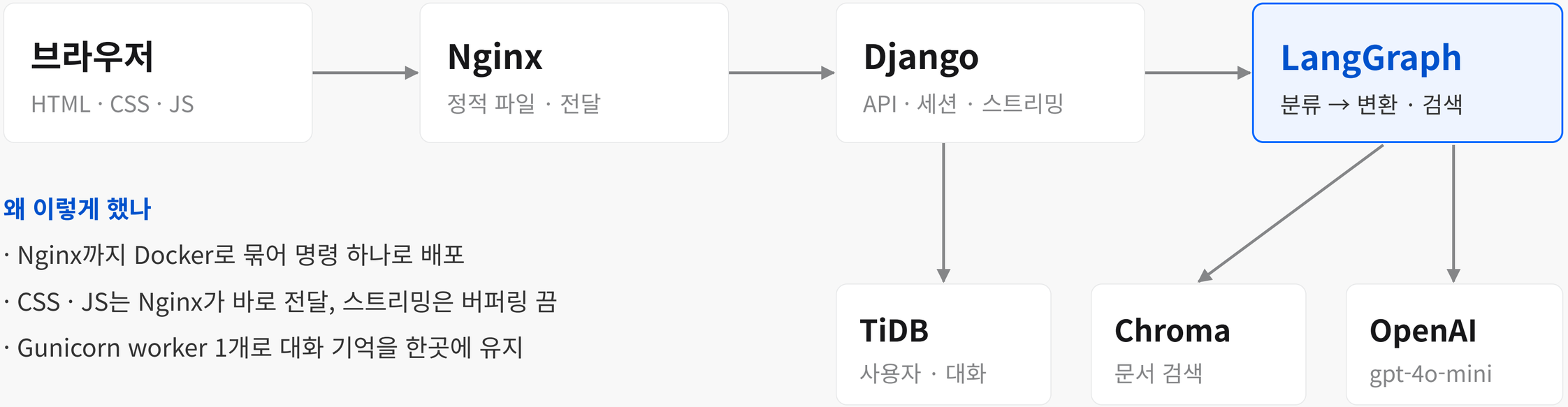
03 · 바뀐 기술

3차 → 4차, 바뀐 것만

항목	3차	4차
화면	Streamlit	Django 템플릿 + HTML · CSS · JS
백엔드	화면에서 그래프 직접 호출	Django API (가입 · 로그인 · 대화 · 채팅)
진행 표시	Streamlit이 화면을 다시 그림	서버가 단계마다 한 줄씩 전송 (스트리밍)
로그인	아이디만	아이디 + 비밀번호 (해시 저장)
대화 관리	목록 · 다시 열기	목록 · 다시 열기 · 삭제
질문 분류	LLM이 판단	코드 규칙 먼저 + LLM, 코드 없으면 안내
평가	코드 변환	코드 변환 + RAG 평가 (RAGAS)
배포	Streamlit Cloud	Docker → AWS EC2

03 · 바뀐 기술 · 배포 설계

앱 하나에서, 역할별로 나눈 구성으로



왜 이렇게 했나

- Nginx까지 Docker로 묶어 명령 하나로 배포
- CSS · JS는 Nginx가 바로 전달, 스트리밍은 버퍼링 끔
- Gunicorn worker 1개로 대화 기억을 한곳에 유지

03 · 바뀐 기술 · Django API

앱 두 개로 나누고, API로 연결했습니다

users

사용자 관리
회원가입 · 로그인 · 로그아웃

POST	/api/users/signup/	회원가입
POST	/api/users/login/	로그인
POST	/api/users/logout/	로그아웃

langration

질문 파이프라인 수행 · 답변 생성
대화 세션 관리

POST	/api/langration/sessions/	새 대화
GET	/api/langration/sessions/	대화 목록
GET	/api/langration/sessions/<id>/	대화 불러오기
DELETE	/api/langration/sessions/<id>/	대화 삭제
POST	/api/langration/chat/	LLM 질문 (스트리밍)

03 · 바뀐 기술 · 질문 분류

분류는 코드로 먼저 확인합니다

① 규칙 먼저

코드 · 옛 import는 코드로 판정

3차 모든 질문을 LLM이 판단

4차 코드 · 옛 import · 옛 API 이름을 먼저 코드로 확인 (규칙 R1 · R2)
나머지만 LLM이 판단

② 코드 없으면 안내

고칠 코드가 없으면 변환하지 않음

3차 코드 없이도 변환을 시도

4차 코드가 없으면 변환하지 않고 코드를 붙여 달라고 안내 (no_code)

③ 실패 기록

분류 실패를 따로 남김

3차 실패해도 일반 답변으로 넘어감

4차 분류 오류를 route_error로 기록해 평가에서도 실패로 셈
일반 답변 온도 0.7 → 0.3

03 · 바뀐 기술 · 진행 표시

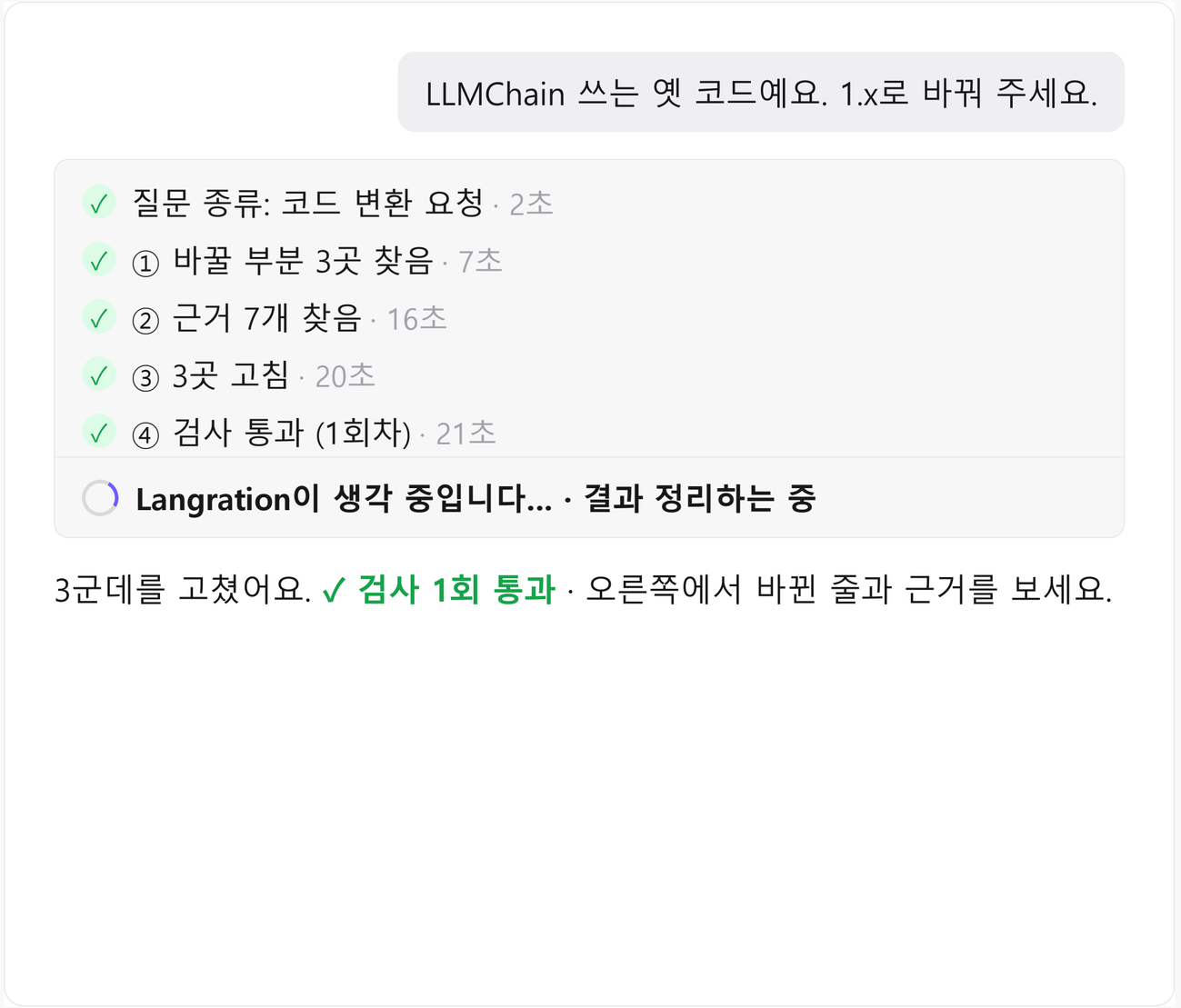
같은 진행 표시, 보여 주는 방식이 바뀌었습니다

3차 · Streamlit

화면과 그래프가 한 프로그램 안에 있어
단계가 끝나면 Streamlit이 화면을 다시 그림

4차 · 서버 → 브라우저

Django가 단계마다 한 줄씩 전송 (NDJSON)
Nginx는 모으지 않고 바로 전달 (버퍼링 끔)
화면은 도착한 줄을 차례로 쌓음



04 · 시연

배포 주소에서 바로 보겠습니다

http://16.176.158.134/

1

가입 · 로그인

아이디 + 비밀번호

2

예제 보내기

옛 코드 한 개

3

진행 단계 도착

한 줄씩 쌓이는
진행 상자

4

결과 패널

전후 비교
근거 · 검사

5

대화 관리

다시 열기 · 삭제

05 · 결과 정리

같은 모델로, 3차보다 모든 지표가 올랐습니다

최종 검증 통과

3차 53%

↓

93%

+40%p

재시도 안에 검사를 통과한 비율

공통 기준 통과

3차 57%

↓

63%

+6%p

정답 import까지 모두 맞음
일반 LLM 0%

자체 평가 21항목

3차 83.3

↓

91.8점

B → A

분류 · 변환 · 검증 · 화면

평가 범위

3차 코드 변환

↓

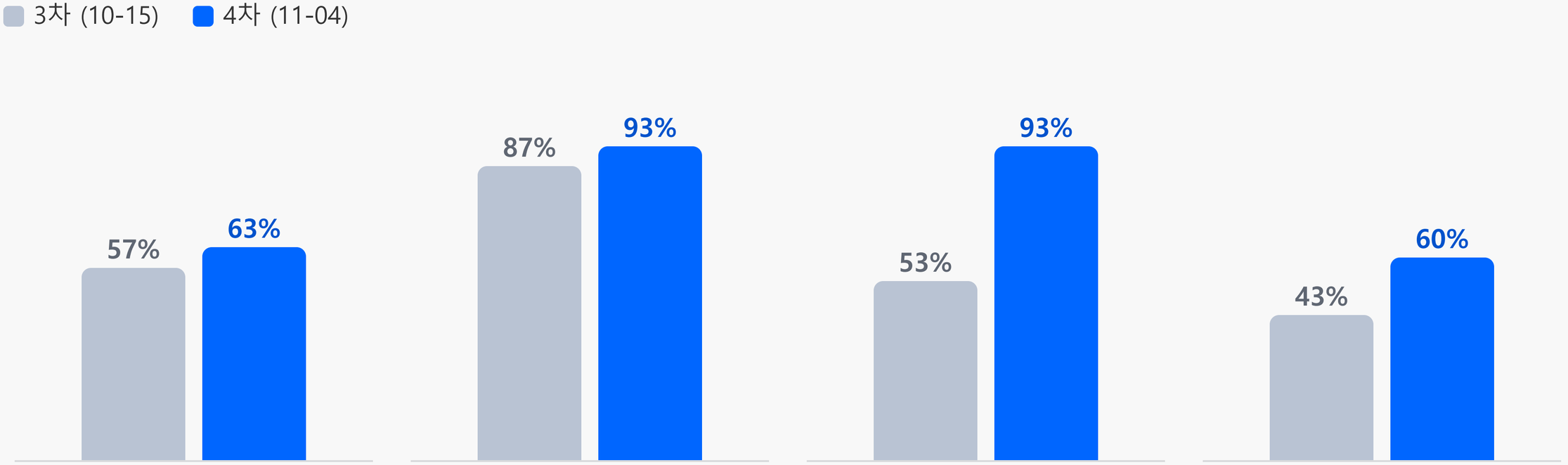
+ RAG 답변

RAGAS 추가

문서 질의응답 30문항

05 · 결과 정리 · 코드 변환

3차 → 4차, 지표별로 보면



공통 기준 통과
실행 기준 + 정답 import까지 모두 맞은 문항

실행 기준 통과
문법 · 옛 import 없음 · import 존재 · 결과값 보존 (정답 import 제외)

최종 검증 통과
최대 3회 재시도 안에 검사를 통과한 문항

1회차 검증 통과
재시도 없이 첫 시도에 검사를 통과한 문항

05 · 결과 정리 · RAG 평가

3차는 코드 변환만, 4차는 RAG 답변까지

3차 평가

✓ 코드 변환

✗ 문서 질의응답은 재지 않음

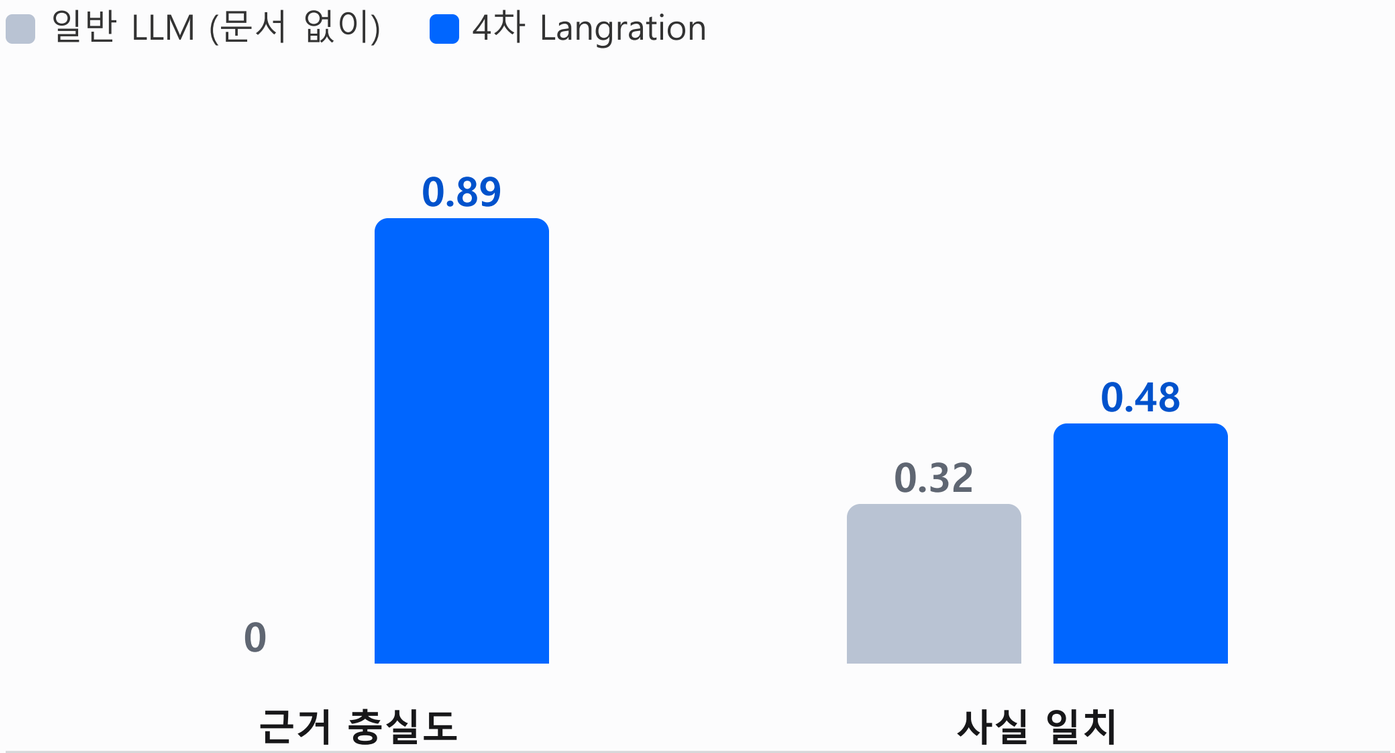
4차 평가

✓ 코드 변환 (다시 측정)

✓ 문서 질의응답 RAGAS 30문항

근거 충실도 찾은 문서 내용으로만 답했는가

사실 일치 정답과 사실이 맞는가



감사합니다

질문 받겠습니다